

# Le Pipeline PowerShell et la Manipulation d'Objets

## 1. Fondations et Architecture du Pipeline Orienté Objet

Dans les interpréteurs de commandes historiques (shells Unix ou cmd.exe), le pipeline (|) transfère des flux d'octets ou de texte brut entre deux programmes. Cette sérialisation textuelle impose de redécouper les lignes, de gérer des séparateurs variables et de subir les aléas de formatage de l'interface système.

PowerShell transforme ce mécanisme en implémentant un **pipeline d'objets en mémoire** :

[Producteur d'Objets] ---> (Instance .NET en mémoire) ---> [Consommateur / Filtre] ---> (Objet projeté) ---> [Export / Formatage]

### 1.1 Principes fondamentaux du transport d'objets

1. **Intégrité de la structure** : Chaque commande émet une ou plusieurs instances d'objets fortement typées. Toutes les métadonnées (attributs, types de données, méthodes) restent actives d'un bout à l'autre de la chaîne d'exécution.
2. **Traitement au fil de l'eau (Streaming)** : PowerShell ne charge pas nécessairement l'intégralité d'une collection en mémoire avant de l'envoyer au maillon suivant. Les objets traversent le pipeline un par un dès qu'ils sont instanciés, réduisant la consommation de mémoire vive.
3. **La variable automatique \$\_ (ou \$PSItem)** : Dans les blocs de script du pipeline, la variable \$\_ représente le conteneur dynamique de l'objet actuellement acheminé dans le tube.

## 2. Découverte et Cartographie des Propriétés (Get-Member)

Avant d'appliquer une transformation ou une condition sur un jeu de données, l'administrateur système doit inspecter le schéma interne de l'objet manipulé afin d'identifier les attributs disponibles.

### 2.1 Inspection via Get-Member

La cmdlet Get-Member (alias : gm) analyse le type .NET de l'objet reçu via le pipeline et dresse

l'inventaire de ses membres.

Get-Process | Get-Member

Pour limiter l'affichage aux seules propriétés de données (en masquant les méthodes d'action), on utilise le paramètre -MemberType :

Get-Process | Get-Member -MemberType Property, NoteProperty

## 2.2 Analyse des objets du système

### Analyse de Get-Process

- **Classe .NET** : System.Diagnostics.Process
- **Propriété de consommation processeur** : CPU (type System.Double). Elle représente le temps processeur cumulé consommé par le processus depuis son initialisation, exprimé en secondes.
- **Propriété d'identification système** : Id (type System.Int32). Représente le PID numérique unique attribué par le noyau du système d'exploitation.

### Analyse de Get-Service

- **Classe .NET** : System.ServiceProcess.ServiceController
- **Propriété d'état opérationnel** : Status (type énuméré System.ServiceProcess.ServiceControllerStatus). Cette propriété prend des états finis stricts tels que Running (En cours d'exécution), Stopped (Arrêté), ou Paused (Suspendu).

## 3. Filtrage de Données dans le Flux (Where-Object)

Le filtrage d'un flux d'objets permet de restreindre le traitement aux seuls enregistrements répondant à une assertion logique.

### 3.1 Syntaxe classique vs Syntaxe simplifiée

Il existe deux manières de formuler un filtre avec Where-Object (alias : where ou ?) :

#### Syntaxe avec bloc de script (Standard historique)

Recommandée pour les conditions complexes combinant plusieurs clauses (-and, -or) :

Get-Service | Where-Object { \$\_.Status -eq "Stopped" }

#### Syntaxe simplifiée (Introduite avec PowerShell 3.0)

Recommandée pour les comparaisons directes à propriété unique :

Get-Service | Where-Object Status -eq "Stopped"

## 3.2 Catalogue des opérateurs de comparaison

Les opérateurs de comparaison en PowerShell sont insensibles à la casse par défaut.

| Opérateur | Signification mathématique / logique               | Exemple                                         |
|-----------|----------------------------------------------------|-------------------------------------------------|
| -eq       | Égal à ( <i>Equal</i> )                            | <code>\$_Status -eq "Running"</code>            |
| -ne       | Différent de ( <i>Not Equal</i> )                  | <code>\$_ProcessName -ne "svchost"</code>       |
| -gt       | Strictement supérieur à ( <i>Greater Than</i> )    | <code>\$_CPU -gt 5</code>                       |
| -ge       | Supérieur ou égal à ( <i>Greater or Equal</i> )    | <code>\$_Id -ge 1000</code>                     |
| -lt       | Strictement inférieur à ( <i>Less Than</i> )       | <code>\$_CPU -lt 1</code>                       |
| -le       | Inférieur ou égal à ( <i>Less or Equal</i> )       | <code>\$_CPU -le 0.5</code>                     |
| -like     | Correspondance textuelle avec motifs jokers (*, ?) | <code>\$_Name -like "W*"</code>                 |
| -notlike  | Non-correspondance textuelle avec motifs jokers    | <code>\$_Name -notlike "win*"</code>            |
| -match    | Correspondance via Expression Régulière (Regex)    | <code>\$_ProcessName -match "[a-z]{3}\$"</code> |

## 3.3 Cas pratiques de filtrage

### 1. Isolation des services arrêtés

Get-Service | Where-Object Status -eq "Stopped"

## 2. Isolation des processus à charge CPU significative

Get-Process | Where-Object CPU -gt 5

## 3. Ciblage nominatif précis d'un exécutable

Get-Process | Where-Object ProcessName -eq "svchost"

# 4. Ordonnancement et Projection : Sort-Object et Select-Object

Après avoir isolé les éléments nécessaires, il convient de structurer leur disposition et de ne conserver que les champs stratégiques.

### 4.1 Tri séquentiel des données (Sort-Object)

La cmdlet Sort-Object (alias : sort) réordonne les objets selon une ou plusieurs propriétés.

- **Tri croissant (comportement par défaut) :**  
Get-Process | Sort-Object -Property CPU
- **Tri décroissant (du plus grand au plus petit) :**  
Get-Process | Sort-Object -Property CPU -Descending

*Remarque technique sur les performances :* Contrairement aux filtres qui traitent les éléments au fil de l'eau, Sort-Object est une opération bloquante (*blocking cmdlet*). Elle doit charger l'ensemble des objets en mémoire pour déterminer leur ordre relatif avant d'émettre le premier résultat vers le maillon suivant.

### 4.2 Projection de colonnes et extraction (Select-Object)

La cmdlet Select-Object (alias : select) remplit deux fonctions distinctes : la projection verticale (choix des propriétés) et la projection horizontale (nombre d'éléments).

#### Sélection de colonnes utiles

Par défaut, PowerShell applique un filtre de rendu visuel prédéfini. Pour forcer l'extraction de propriétés cibles :

Get-Process | Select-Object -Property ProcessName, Id, CPU

#### Limitation volumétrique

Le commutateur -First permet de borner le nombre d'enregistrements retournés, interrompant

le pipeline dès que le quota est atteint :

```
Get-Process | Sort-Object -Property CPU -Descending | Select-Object -Property  
ProcessName, Id, CPU -First 5
```

## 5. Persistance des Données : Export-Csv

L'extraction de rapports exploitables impose d'enregistrer les données dans un format standardisé sans dénaturer la granularité des colonnes.

### 5.1 Différence entre Export-Csv et Out-File

- **Out-File** ou la redirection > enregistre le texte brut tel qu'il est dessiné dans la console, avec des colonnes potentiellement tronquées par des points de suspension (...) et des espaces arbitraires.
- **Export-Csv** analyse les objets .NET transmis dans le pipeline, extrait le nom des propriétés pour concevoir la ligne d'en-tête, et insère les valeurs typées ligne par ligne.

### 5.2 Paramètres critiques en entreprise

```
Export-Csv -Path "$HOME\Desktop\rapport.csv" -NoTypeInfoInformation -Delimiter ";" -Encoding  
UTF8
```

- **-NoTypeInfoInformation** : Supprime la première ligne d'en-tête technique (#TYPE System.Diagnostics.Process) qui perturbe l'ingestion du CSV par des outils tiers ou des tableurs.
- **-Delimiter ";"** : Recommandé pour garantir une ouverture native dans les versions francophones de tableurs sans nécessiter de conversion textuelle.
- **-Encoding UTF8** : Garantit la conservation des accents et des caractères internationaux dans les libellés (DisplayName).

## 6. Étude de Cas : Conception d'un Pipeline Industriel

### 6.1 Énoncé du cas de synthèse

Produire un rapport au format CSV sur le bureau de l'utilisateur, recensant le nom d'affichage et l'état de l'ensemble des services démarrés dont le nom débute par la lettre "W".

### 6.2 Analyse méthodologique étape par étape

1. **Collecte initiale** : Récupérer les services commençant par "W".
2. **Filtrage de conformité** : Éliminer les services arrêtés pour ne conserver que l'état Running.
3. **Projection ciblée** : Restreindre l'objet aux colonnes DisplayName et Status.

4. **Persistence disque** : S rialiser le r sultat au format CSV.

### 6.3 Impl mentation technique

```
Get-Service -Name "W*" | Where-Object Status -eq "Running" | Select-Object -Property  
DisplayName, Status | Export-Csv -Path "$HOME\Desktop\rapport_services_W.csv"  
-NoTypeInfoInformation -Delimiter ";" -Encoding UTF8
```

### 6.4 Optimisation : La R gle « Filtrer   Gauche » (*Filter Left*)

Dans l'impl mentation ci-dessus, observez l'instruction `Get-Service -Name "W*"`.

Il aurait  t  possible d' crire :

# Approche moins performante :

```
Get-Service | Where-Object { $_.Name -like "W*" -and $_.Status -eq "Running" }
```

**Pourquoi l'approche « Filtrer   gauche » est sup rieure :**

- Le param tre `-Name "W*"` demande au fournisseur Windows de ne retourner que les services pertinents d s la source.
- L'approche sans param tre force PowerShell   charger en m moire l'int gralit  des centaines de services du syst me d'exploitation, pour en  liminer la majorit  dans le pipeline.
- **R gle g n rale** : Tout ce qui peut  tre filtr  directement par la commande source via ses param tres natifs doit l' tre avant d'entrer dans un tube `Where-Object`.

## 7. Synth se des Commandes et Encha nements Piliers

| R le op rationnel | Commande type | Exemple d'encha nement                                    |
|-------------------|---------------|-----------------------------------------------------------|
| Inspecter         | Get-Member    | Get-Process   Get-Member<br>-MemberType Property          |
| Filtrer           | Where-Object  | Get-Service  <br>Where-Object Status -eq<br>"Running"     |
| Trier             | Sort-Object   | Get-Process   Sort-Object<br>-Property CPU<br>-Descending |
| Projeter          | Select-Object | Get-Process                                               |

|                  |            |                                                                         |
|------------------|------------|-------------------------------------------------------------------------|
|                  |            | Select-Object<br>ProcessName, Id -First 10                              |
| <b>Persister</b> | Export-Csv | ...   Export-Csv -Path<br>"C:\Audit\data.csv"<br>-NoTypeInfoInformation |